01 / THE THESIS

Security for software built with AI.

+

Find bugs.
Prove correctness.

A CLI for AI agents.
A dashboard for humans.

h5i.dev ↗

01 / AI-LED RED-TEAMING

02 / FORMAL VERIFICATION

02 / THE FOUNDING INSIGHT

Find bugs. Prove correctness. Two sides of the same coin.

A lesson from my PhD: secure software needs both.

01 / FIND BUGS

Discover what we missed.

Explore the running app to uncover vulnerabilities, including cases the specification never anticipated.

FINDINGS REVEAL GAPS IN OUR RULES

02 / PROVE CORRECTNESS

Establish what must hold.

Prove stated properties for every case covered by the model, under explicit assumptions.

PROOFS ESTABLISH THE RULES WE STATE

> Find bugs to challenge our understanding. Prove correctness to make it precise. We need both.

03 / THE PRODUCT

One workspace. Both sides of web security.

01 / AI-LED RED-TEAMING

AI tests. You stay in control.

An AI-friendly CLI runs the tests. A dashboard lets you follow the agent, review findings and manage browser sessions.

AI OPERATES THE CLI ↔ YOU OVERSEE THE DASHBOARD

02 / FORMAL VERIFICATION

Prove the critical rules.

Use h5i-app and Lean 4 to check stated properties of extracted application logic.

OUTPUT → THEOREMS + RECEIPT

Testing works with existing apps. The Rust framework is optional. Proofs apply to stated properties under documented assumptions.

04 / FIND VULNERABILITIES

The agent runs the tests. You see and manage the work.

Two interfaces to the same session: commands for the AI, oversight for you.

AI AGENT / CLI 01

Easy for the agent to operate.

Focused commands, compact output and reusable evidence IDs.

```
$ h5i websec replay req_42 \
    --set query.id=1042
  → res_43 · HTTP 200
$ h5i websec diff res_42 res_43
  owner: Alice → Bob
```

same session



HUMAN / DASHBOARD 02

Easy for you to oversee.

checkout-idor	working
01 / BROWSE	Agent opened Alice's orders
02 / REPLAY	Agent changed the order ID

FINDING / AUTHORIZATION

Alice can read Bob's order.

Evidence: req_42 · res_43

Take browser control | Stop browser

05 / TECHNICAL DEPTH

Confidence comes with a chain of evidence.

BOUNDED EXECUTION

Set the agent's reach.

Optional boxes constrain files, network access and runtime. Receipts distinguish host observations from box claims.

POLICY → EXECUTION → RECEIPT

Network enforcement depends on the tier. Only microvm provides a separate guest kernel.

CHECKED PROPERTIES

Tie proofs to the logic.

Charon and Aeneas extract Rust into Lean. Verification gates check proof builds, extraction drift, axioms and mutations.

RUST → LEAN → VERIFICATION

The specification, translation and runtime assumptions define the guarantee.

Box policies ↗ Proof scope & assumptions ↗

06 / WHY H5I

Built for AI to operate.
Built for humans to oversee.

h5i's advantage is the connection between an agent-friendly CLI and a dashboard for human understanding and control.

DESIGNED FOR	WHAT THIS MAKES POSSIBLE
The AI doing the work	Run focused commands and use compact results to choose the next test.
The human overseeing it	Follow actions, inspect findings, and take control of or stop the browser.
Their shared workflow	Connect the agent's actions to the session and evidence you review.

> The product value: AI can work through the tests, and you can understand and manage what it does.

07 / EARLY TRACTION

Built in the open.
Early developer interest.

1.6k+

RELEASE DOWNLOADS

700+

GITHUB STARS

Open-source interest is the starting signal.
The next milestone is repeated use on real applications.

OPEN DEVELOPMENT

Apache 2.0 · CLI + review console
Browser testing + Rust verification

Maintainer-reported · October 11, 2026 Repository ↗ Releases ↗

08 / WHAT COMES NEXT

Find bugs. Prove correctness. Build trust.

Next: work with design partners on real applications, improve the workflow, and build toward the first paid pilots.

Start a conversation ↗

Explore h5i ↗